CO3_AT3_MOCK_INTERVIEW

View Only

← Back

QUESTIONS

- Q1 Find 10 marks
- Q2 Modification 10 marks
- Q3 Identify 10 marks
- Q4 Output 10 marks
- Q5 Operator 10 marks
- Q6 Output 10 marks
- Q7 Destructor 10 marks
- Q8 Virtual 10 marks
- Q9 Modification 10 marks
- Q10 Output 10 marks

10/10 answered

Q1 Find

10 marks

What happens to the managed object?

Your Answer

The managed object remains alive as long as an owning smart pointer exists. When the last owner is destroyed or reset, the managed object is automatically deleted and its memory is released.

CO3_AT3_MOCK_INTERVIEW

View Only

← Back

QUESTIONS

- Q1 Find 10 marks
- Q2 Modification 10 marks
- Q3 Identify 10 marks
- Q4 Output 10 marks
- Q5 Operator 10 marks
- Q6 Output 10 marks
- Q7 Destructor 10 marks
- Q8 Virtual 10 marks
- Q9 Modification 10 marks
- Q10 Output 10 marks

10/10 answered

Q2 Modification

10 marks

What modification resolves without removing inheritance? class A { public: void show() { cout << "A"; } }; class B { public: void show() { cout << "B"; } }; class C : public A, public B {}; int main() { C obj; obj.show(); }

Your Answer

The call is ambiguous because both A and B contain show(). Resolve it using scope resolution:
obj.A::show(); // Prints A
Alternatively, use obj.B::show(); to print B

CO3_AT3_MOCK_INTERVIEW

View Only

← Back

QUESTIONS

- Q1 Find 10 marks
- Q2 Modification 10 marks
- Q3 Identify 10 marks
- Q4 Output 10 marks
- Q5 Operator 10 marks
- Q6 Output 10 marks
- Q7 Destructor 10 marks
- Q8 Virtual 10 marks
- Q9 Modification 10 marks
- Q10 Output 10 marks

10/10 answered

Q3 Identify

10 marks

Identify the ownership behavior. shared_ptr<int> p1 = make_shared<int>(10); shared_ptr<int> p2 = p1; p1.reset();

Your Answer

Initially, p1 and p2 share ownership of the same integer, so the reference count is 2. After p1.reset(), p1 becomes empty, but the object is not deleted because p2 still owns it. The object is deleted when p2 is destroyed or reset.

CO3_AT3_MOCK_INTERVIEW

View Only

← Back

QUESTIONS

- Q1 Find 10 marks
- Q2 Modification 10 marks
- Q3 Identify 10 marks
- Q4 Output 10 marks
- Q5 Operator 10 marks
- Q6 Output 10 marks
- Q7 Destructor 10 marks
- Q8 Virtual 10 marks
- Q9 Modification 10 marks
- Q10 Output 10 marks

10/10 answered

Q4 Output

10 marks

What is the output? class A { public: A() { cout << "A "; } A(const A&) { cout << "C "; } A& operator=(const A&) { cout << "E "; return *this; } }; int main() { A a; A b = a; A c; c = a; }

Your Answer

The output is:
ACAE
a calls the constructor, b = a calls the copy constructor, c calls the constructor, and c = a calls the assignment operator.

CO3_AT3_MOCK_INTERVIEW

View Only

← Back

QUESTIONS

- Q1 Find 10 marks
- Q2 Modification 10 marks
- Q3 Identify 10 marks
- Q4 Output 10 marks
- Q5 Operator 10 marks
- Q6 Output 10 marks
- Q7 Destructor 10 marks
- Q8 Virtual 10 marks
- Q9 Modification 10 marks
- Q10 Output 10 marks

10/10 answered

Q5 Operator

10 marks

Which increment operation does this represent, and what would you change for postfix counter++?

Your Answer

It represents the prefix increment operator because it has no dummy parameter:
Counter& operator++();
For postfix counter++, add a dummy int parameter:
Counter operator++(int);
Postfix should return the old value.

CO3_AT3_MOCK_INTERVIEW

View Only

← Back

QUESTIONS

- **Q1** Find 10 marks
- **Q2** Modification 10 marks
- **Q3** Identify 10 marks
- **Q4** Output 10 marks
- **Q5** Operator 10 marks
- **Q6** Output 10 marks
- **Q7** Destructor 10 marks
- **Q8** Virtual 10 marks
- **Q9** Modification 10 marks
- **Q10** Output 10 marks

10/10 answered

Q6 Output

10 marks

Predict the result and identify the memory-management problem. class A { int* p; public: A(int x) : p(new int(x)) {} ~A() { delete p; } }; int main() { A a(10); A b(a); }

Your Answer

The default copy constructor performs a shallow copy, so both a.p and b.p point to the same memory. During destruction, both objects try to delete the same pointer. This causes double deletion and undefined behaviour, possibly a runtime crash.

CO3_AT3_MOCK_INTERVIEW

View Only

← Back

QUESTIONS

- **Q1** Find — 10 marks
- **Q2** Modification — 10 marks
- **Q3** Identify — 10 marks
- **Q4** Output — 10 marks
- **Q5** Operator — 10 marks
- **Q6** Output — 10 marks
- **Q7** Destructor — 10 marks
- **Q8** Virtual — 10 marks
- **Q9** Modification — 10 marks
- **Q10** Output — 10 marks

10/10 answered

Q7 Destructor

10 marks

Determine the exact destruction order. class Employee { public: Employee() {} ~Employee() { cout << "Employee "; } }; class Manager { Employee e1; Employee e2; public: ~Manager() { cout << "Manager "; } };

Your Answer

The Manager destructor body executes first. Then its members are destroyed in reverse declaration order: e2 followed by e1.
Manager Employee Employee

CO3_AT3_MOCK_INTERVIEW

View Only

← Back

QUESTIONS

- Q1 Find 10 marks
- Q2 Modification 10 marks
- Q3 Identify 10 marks
- Q4 Output 10 marks
- Q5 Operator 10 marks
- Q6 Output 10 marks
- Q7 Destructor 10 marks
- Q8 Virtual 10 marks
- Q9 Modification 10 marks
- Q10 Output 10 marks

10/10 answered

Q8 Virtual

10 marks

What changes if virtual is removed from Base::show() in [class Base { public: virtual void show() { cout << "Base"; } }; class Derived : public Base { public: void show() override { cout << "Derived"; } }; int main() { Base* p = new Derived(); p->show(); }]

Your Answer

If only virtual is removed while override remains, the program gives a compilation error because Derived::show() no longer overrides a virtual function. If override is also removed, static binding occurs and p->show() calls Base::show(), producing:
Base

CO3_AT3_MOCK_INTERVIEW

View Only

← Back

QUESTIONS

- Q1 Find 10 marks
- Q2 Modification 10 marks
- Q3 Identify 10 marks
- Q4 Output 10 marks
- Q5 Operator 10 marks
- Q6 Output 10 marks
- Q7 Destructor 10 marks
- Q8 Virtual 10 marks
- Q9 Modification 10 marks
- Q10 Output 10 marks

10/10 answered

Q9 Modification

10 marks

Write the minimum modification required to make Question safe for copying. class A { int* p; public: A(int x) : p(new int(x)) {} ~A() { delete p; } }; int main() { A a(10); A b(a); }

Your Answer

Add a deep-copy constructor:
A(const A& other) : p(new int(*other.p)) {}
This allocates separate memory for the copied object and prevents double deletion.

CO3_AT3_MOCK_INTERVIEW

View Only

← Back

QUESTIONS

- **Q1** Find 10 marks
- **Q2** Modification 10 marks
- **Q3** Identify 10 marks
- **Q4** Output 10 marks
- **Q5** Operator 10 marks
- **Q6** Output 10 marks
- **Q7** Destructor 10 marks
- **Q8** Virtual 10 marks
- **Q9** Modification 10 marks
- **Q10** Output 10 marks

10/10 answered

Q10 Output

10 marks

What is the output? class A { public: A() { cout << "A "; } }; class B : public A { public: B() { cout << "B "; } }; class C : public B { public: C() { cout << "C "; } }; int main() { C obj; }

Your Answer

Constructors execute from the base class to the derived class: A, then B, then C.
ABC